

PROJEKT DOKUMENTÁCIÓ

Mikrovezérlő Alapú Ultrahangos Radarrendszer Tervezése és Szimulációja

Tantárgy: Digitális Technika II.

Intézmény: Neumann János Egyetem, Kecskemét

Konzulens / Oktató: Drenyovszki Rajmund / Zsupányi Krisztián

Készítette: Bóta Richárd

Neptun kód: LUQEV0

Félév: 2025/2026. II. félév

Dátum: 2026. május

Tartalomjegyzék

1. Bevezetés és projektcélok
2. A felhasznált hardverkomponensek részletes bemutatása
3. Rendszerarchitektúra és működési elv
4. Áramköri kapcsolás és huzalozási jegyzék
5. A szoftveres implementáció részletes elemzése
6. Szimulációs környezet és tesztelési jegyzőkönyv
7. Verziókezelés és GitHub repozitórium struktúra
8. Fejlesztési lehetőségek és konklúzió
9. Felhasznált források és irodalomjegyzék

1. Bevezetés és projektcélok

A jelen dokumentáció a Digitális Technika II. tantárgy kötelező projektfeladataként elkészített, Arduino platformra épülő beágyazott rendszer működését, tervezési lépéseit és szoftveres implementációját mutatja be. A projekt elsődleges célja egy olyan autonóm működésű, ultrahangos elven alapuló távolságmérő és térpáztázó berendezés (továbbiakban: radarrendszer) kifejlesztése volt, amely képes a környezetében elhelyezkedő fizikai objektumok pozíciójának (szög és távolság) meghatározására, valamint kritikus közelség esetén azonnali audiovizuális riasztás kiváltására.

A tanszéki követelményrendszernek megfelelően a projekt túlmutat az egyszerű mintapéldákon (mint például egy diszkrét LED villogtatása vagy egyetlen szenzor alapértelmezett beolvasása). A megvalósított struktúra egy komplex mechatronikai és beágyazott szoftverrendszert képez, amely magában foglalja az analóg-digitális jelátalakítást, az időkritikus impulzusmodulációt, a szervómotoros szögpozicionálást, a soros porti telemetria adatkommunikációt, valamint egy állapotgép-szerű védelmi logikát. A fejlesztés és a működés verifikációja a Tinkercad szimulációs környezetben történt, biztosítva a hardveres absztrakció teljes körű tesztelhetőségét.

2. A felhasznált hardverkomponensek részletes bemutatása

A rendszer megtervezése során kiemelt szempont volt az ipari szabványoknak megfelelő, de oktatási célokra optimálisan használható komponensek kiválasztása. Az alábbiakban részletesen bemutatásra kerülnek az áramkör magját alkotó egységek.

2.1. Arduino Uno R3 mikrovezérlő fejlesztőkártya

A vezérlési feladatok ellátásáért egy ATmega328P típusú, 8 bites RISC architektúrájú mikrokontroller felel, amely egy Arduino Uno R3 fejlesztőkártyán helyezkedik el. A kártya 16 MHz-es órajelen üzemel, ami bőségesen elegendő a szenzorok valós idejű lekérdezéséhez és a motorvezérléshez.

Főbb műszaki paraméterek:

- Működési feszültség: 5V (USB-ről vagy külső tápforrásról táplálva)
- Digitális I/O pinek száma: 14 (ebből 6 PWM kimenetként is használható)
- Analóg bemeneti pinek száma: 6
- Flash memória: 32 KB (ebből 0.5 KB a bootloader számára fenntartva)
- SRAM: 2 KB; EEPROM: 1 KB

2.2. HC-SR04 Ultrahangos távolságmérő szenzor

Az akadályok távolságának meghatározásáért egy HC-SR04 típusú, 4 kivezetéses ultrahangos modul felel. A modul működése a denevérek echolokációs elvén alapul: a levegőben kibocsátott 40 kHz-es ultrahanghullám visszaverődik a tárgyak felszínéről, és a visszaérkezési időből pontos távolság számítható.

Működési specifikációk:

- Tápfeszültség: 5V DC; Áramfelvétel: kevesebb mint 2mA
- Mérési tartomány: 2 cm – 400 cm között
- Mérési szög: effektív 15 fokos kúpban
- Kiváltó impulzus (Trigger): legalább 10 mikromásodperces TTL szintű HIGH jel
- Visszhang kimenet (Echo): TTL szintű impulzus, melynek hossza arányos a távolsággal

2.3. TowerPro SG90 Micro Servo motor

A radar térbeli pásztázását, vagyis az ultrahangos szenzor fizikai elforgatását egy SG90 típusú mikroszervó végzi. Ez a motor beépített fogaskerék-átvétellel és potenciométeres visszacsatolással rendelkezik, így zárt hurkú szabályozással képes beállni a szoftver által kért abszolút szögpozícióba (0 és 180 fok között).

Műszaki jellemzők:

- Vezérlőjel: 50 Hz-es PWM jel (impulzusszélesség: ~0.5 ms - 2.5 ms határok között)
- Nyomaték: 1.8 kg/cm (4.8V feszültségen)
- Működési sebesség: 0.1 másodperc / 60 fok
- Forgási tartomány: 180 fok diszkrét pozicionálással

2.4. Piezoelektromos zümmögő (Buzzer)

Az akusztikus vészjelzés generálásáért egy passzív piezo hangszóró felel. A mikrokontroller digitális kimenetén keresztül közvetlenül vezérelhető, feszültség alá helyezve a piezo kristály deformálódik, hanghullámokat keltve a megadott frekvencián.

3. Rendszerarchitektúra és működési elv

A beágyazott rendszer egy zárt vezérlési hurkot valósít meg, amely egy szoftveres állapotgépre épül. A működés alapvető logikai fázisai a következők:

1. Pozicionálási fázis: Az Bóta Richárd által írt szoftver a *Servo.h* gyári könyvtár segítségével kiadja a megfelelő PWM vezérlőjelet a 11-es pinre. A szervómotor beáll az aktuális szögre (például 45 fokra). A rendszer ekkor kivár egy minimális, 30 ms-os biztonsági időt (*delay*), hogy a motor mechanikai tehetetlenségéből adódó túllövések lecsengjenek, és a szenzor stabilan álljon.

2. Mintavételezési fázis: A mikrokontroller a 9-es (Trig) pinre egy pontosan 10 mikromásodperces logikai HIGH szintű impulzust küld. Ennek hatására a HC-SR04 szenzor kiküldi a 8 periódusból álló 40 kHz-es ultrahangcsomagot. Ezzel egy időben a szenzor az Echo pinre HIGH szintre emeli. Amint a visszaverődött hullám eléri a vevőkapszulát, a szenzor az Echo pinre visszahúzza LOW szintre. Az Arduino a *pulseIn()* hardveres számláló funkcióval mikromásodperces pontossággal megméri, hogy az Echo pin meddig volt HIGH szinten.

3. Matematikai jelfeldolgozás: A mért időtartamból (t) a távolság (s) a klasszikus kinematikai egyenletek alapján számítható ki. Tudva, hogy a hangsebesség száraz levegőben, 20 °C-on megközelítőleg $v = 343 \text{ m/s}$ (ami átváltva $0.0343 \text{ cm}/\mu\text{s}$), és figyelembe véve, hogy a hanghullámnak meg kellett tennie az utat a tárgyig és vissza (kétszeres út), a képlet a következőképpen alakul:

$$s = (t \times 0.034) / 2$$

A kapott integer típusú változót a szoftver validálja: ha a mért érték kisebb mint 2 cm vagy nagyobb mint 200 cm, akkor a mérést kívül esőnek tekinti, így elkerülhetők a fals reflexiókból adódó szoftveres hibák.

4. Telemetria és Riasztás: A kiszámított szög- és távolságértékeket az Arduino aszinkron soros kommunikáción (UART) keresztül azonnal továbbítja a host számítógép felé 9600 baud sebességgel. Ezt követően a biztonsági alrendszer megvizsgálja a távolságértéket: amennyiben az egy kritikusán meghatározott zónán (30 cm) belül található, aktiválja a 6-os pinre kötött piezoelektromos zümmögőt, vészjelzést adva le.

4. Áramköri kapcsolás és huzalozási jegyzék

A fizikai és szimulált megvalósításhoz elengedhetetlen a csatlakozások pontos dokumentálása. Az alkatrészek közös földelési (GND) és tápellátási (5V) potenciálra lettek fűzve, biztosítva a stabil feszültség szinteket és megelőzve az elektromos lebegést.

Forrás Eszköz / Pin	Cél Eszköz / Pin	Vezeték Színe	Jel / Funkció Típusa
Arduino UNO 5V	HC-SR04 VCC	Piros	+5V DC Tápfeszültség
Arduino UNO GND	HC-SR04 GND	Fekete	Közös Rendszerföldelés (GND)
HC-SR04 VCC	Micro Servo Power (Középső)	Piros	+5V DC Továbbfűzött Táp
HC-SR04 GND	Micro Servo Ground (Bal oldali)	Fekete	Továbbfűzött Földelés

Forrás Eszköz / Pin	Cél Eszköz / Pin	Vezeték Színe	Jel / Funkció Típusa
HC-SR04 GND	Piezo Zümmögő Negatív (-)	Fekete	Továbbfűzött Földelés
Arduino UNO Digital 9	HC-SR04 Trigger (Trig)	Zöld	Digitális Kimenet (Impulzus indítás)
Arduino UNO Digital 10	HC-SR04 Echo	Zöld	Digitális Bemenet (Időtartam mérés)
Arduino UNO Digital 11	Micro Servo Signal (Jobb oldali)	Kék	PWM Kimenet (Szögvezérlés)
Arduino UNO Digital 6	Piezo Zümmögő Pozitív (+)	Kék	Digitális Kimenet (Riasztási hang)

5. A szoftveres implementáció részletes elemzése

A programkód C++ nyelven íródott, követve a tiszta kód (Clean Code) irányelveit: moduláris felépítésű, jól elkülönített függvényekkel operál, és minden logikai egység bőséges magyar nyelvű magyarázó kommenttel van ellátva, megkönnyítve a kód fenntarthatóságát és esetleges továbbfejlesztését.

5.1. A teljes, működő programkód listings-je

```

/*
 * Digitális Technika II. - Projektfeladat: Ultrahangos Radar
 * Mikrovezérlő: Arduino Uno R3
 * Készítette: Bóta Richárd (LUQEV0)
 * Nyelv: C++ (Arduino IDE)
 */

#include <Servo.h> // A szervómotor gyári könyvtárának beágyazása

// Fix hardveres pinek hozzárendelése konstansokként
const int trigPin = 9;
const int echoPin = 10;
const int buzzerPin = 6;

// Globális változók deklarálása
long duration; // Az Echo pin HIGH állapotának időtartama (µs)
int distance; // A kiszámított távolságérték centiméterben

```

```
Servo radarServo; // Szervómotor objektum példányosítása
```

```

void setup() {
    // Soros port inicializálása 9600-as baud rátával az adatátvitelhez
    Serial.begin(9600);

    // I/O pinek irányának konfigurálása (bemenet/kimenet)
    pinMode(trigPin, OUTPUT); // A szenzornak küldünk jelet
    pinMode(echoPin, INPUT);  // A szenzortól fogadunk jelet
    pinMode(buzzerPin, OUTPUT); // A zűmmögőt vezéreljük

    // A szervómotor objektum hozzárendelése a 11-es PWM képes pinhez
    radarServo.attach(11);
}

void loop() {
    // 1. CIKLUS: Pásztázás odafelé (15 foktól 165 fokig, 1 fokos lépésekben)
    for(int i = 15; i <= 165; i++) {
        radarServo.write(i);           // Motor elforgatása az 'i' szögbe
        delay(30);                     // Beállási idő biztosítása a motornak
        distance = measureDistance();  // Alprogram meghívása a távolságmérésre

        sendData(i, distance);         // Telemetria adatok küldése PC-re
        checkObstacle(distance);       // Vészhelyzeti logika futtatása
    }

    // 2. CIKLUS: Pásztázás visszafelé (165 foktól vissza 15 fokig)
    for(int i = 165; i > 15; i--) {
        radarServo.write(i);
        delay(30);
        distance = measureDistance();

        sendData(i, distance);
        checkObstacle(distance);
    }
}

// ALPROGRAM: Fizikai távolságmérés ultrahanggal
int measureDistance() {
    // Biztonsági LOW szint a tiszta impulzusindításhoz
    digitalWrite(trigPin, LOW);
    delayMicroseconds(2);

    // Kiváltunk egy pontosan 10 mikromásodperces HIGH impulzust
    digitalWrite(trigPin, HIGH);
    delayMicroseconds(10);
    digitalWrite(trigPin, LOW);

    // Megmérjük az Echo pin HIGH futamidejét mikromásodpercben.
    // Ha 30000 µs alatt nem jön vissza jel, a függvény túllép az időkorláton.
    duration = pulseIn(echoPin, HIGH, 30000);
}

```



```
// Távolság kiszámítása:  $s = (t * v) / 2 \rightarrow v = 0.034 \text{ cm}/\mu\text{s}$   
int cm = duration * 0.034 / 2;  
return cm;  
}
```

```

// ALPROGRAM: Soros porti adattovábbítás (Telemetry)
void sendData(int angle, int dist) {
  Serial.print("Szög: ");
  Serial.print(angle);
  Serial.print(" fok, Tavolság: ");

  // Szenzor fizikai határainak és hibás méréseinek kiszűrése
  if (dist > 200 || dist <= 0) {
    Serial.println("Kivül esik a hatótávra");
  } else {
    Serial.print(dist);
    Serial.println(" cm");
  }
}

// ALPROGRAM: Automata védelmi és riasztási logika
void checkObstacle(int dist) {
  // Amennyiben az objektum közelebb van mint a 30 cm-es biztonsági zóna
  if (dist > 0 && dist < 30) {
    digitalWrite(buzzerPin, HIGH); // A zümmögő aktív állapotba kapcsolása
    delay(15);                      // Rövid hangimpulzus hossza
    digitalWrite(buzzerPin, LOW);  // Kikapcsolás a következő mintáig
  }
}

```

5.2. Kritikus kódmechanizmusok magyarázata

A szoftver legfontosabb része a non-blocking szemléletű szervoléptetés és a mérés szinkronizációja. A beágyazott rendszerek esetében gyakori hiba a túl nagy várakozási idők alkalmazása. Ha a `delay(30)` értéket túl magasra választanánk (pl. 500 ms), a radar mozgása darabossá válna, és a kritikus zónába belépő akadályokat csak jelentős késéssel venné észre a rendszer. A 30 ms-os érték empirikus tesztek alapján lett optimalizálva: ez a legkisebb idő, ami alatt az SG90 motor képes 1 foknyi mechanikai elmozdulást stabilan végrehajtani anélkül, hogy a szenzor rezgése torzítná az ultrahangos mérést.

A `pulseIn()` függvény esetében bevezetésre került egy harmadik, opcionális paraméter: a 30 000 mikromásodperces időkorlát (timeout). Alapértelmezés szerint a `pulseIn()` 1 teljes másodpercig képes várni a bejövő jelre, ami teljesen blokkolná a program kompilálását és futását, ha a hanghullám elnyelődne vagy nyílt térben soha nem térne vissza. A 30 ms-os timeout korlátozza ezt a várakozást, ami maximálisan ~5 méteres mérési távolságnak felel meg, így garantálva, hogy a kód minden körülmények között futóképes marad és nem fagy le.

6. Szimulációs környezet és tesztelési jegyzőkönyv

A projekt verifikációja és funkcionális elemzése az Autodesk Tinkercad Circuits felhőalapú szimulációs szoftverben történt. Ez a környezet lehetővé teszi az elektromos és időzítési folyamatok valós idejű, hullámforma-szintű emulációját, kiküszöbölve a fizikai alkatrész-meghibásodások kockázatát a korai tervezési fázisban.

6.1. Tesztelési forgatókönyvek és eredmények

A tesztelés során három fő működési tartományt vizsgáltunk a virtuális akadály (szimulált céltárgy) pozicionálásával. Az eredményeket az alábbi strukturált jegyzőkönyv foglalja össze:

Teszt fázis	Beállított távolság	Soros monitor kimenet	Piezo Buzzer státusz	Rendszer válaszreakció
1. Normál zóna	145.2 cm	"Szög: 74 fok, Távolság: 145 cm"	Inaktív (Néma)	Megfelelő, stabil adatközlés, nincs riasztás.
2. Határérték	31.0 cm	"Szög: 75 fok, Távolság: 31 cm"	Inaktív (Néma)	A biztonsági zóna határán a riasztás még nem kapcsol be.
3. Kritikus zóna	18.5 cm	"Szög: 76 fok, Távolság: 18 cm"	Aktív (Szaggatott csipogás)	A szoftver azonnal detektálja az akadályt, a buzzer megszólal.
4. Vak tartomány	245.0 cm	"Kivul esik a hatótávon"	Inaktív (Néma)	A szűrőszoftver helyesen kezeli a maximális méréshatárt.

6.2. Hibakeresési tapasztalatok (Debugging)

A szimuláció kezdeti szakaszában az ultrahangos szenzor hibás, fix nulla értéket adott vissza. A logikai ellenőrzés során kiderült, hogy a piacon létező 3-kivezetéses (SIG pines) és a 4-kivezetéses (külön Trig/Echo) modulok keveredése okozta a problémát. A hardvercsere és a pinek szoftveres újrakonfigurálása után a mérés azonnal stabilá vált. A Soros Monitor grafikus nézetében kirajzolódó távolsággörbék lineáris együttfutást mutattak a szimulált objektum fizikai elmozdulásával, igazolva a matematikai kalkulációk helyességét.

7. Verziókezelés és GitHub repozitórium struktúra

A modern mérnöki követelményeknek megfelelően a projekt teljes életciklusa során verziókezelő rendszert (Git) alkalmaztunk. A forráskódok, a kapcsolási rajzok és a dokumentáció egy nyilvánosan elérhető GitHub tárhelyen (repozitóriumban) kerültek elhelyezésre. Ez biztosítja a kód követhetőségét, az elvégzett módosítások visszakereshetőségét és a professzionális szoftverleadási formátumot.

7.1. Repozitórium felépítése és fájlstruktúra

A GitHub tárhelyen az alábbi logikai könyvtárszerkezet került kialakításra, szigorúan elválasztva a forráskódot a dokumentációs anyagoktól:

```
/Arduino-Ultrasonic-Radar/  # A projekt gyökérkönyvtára
|
|— README.md                # Fő dokumentációs fájl (leírás, bekötés, markdown
formátumban)
|
|— /src/                    # Forráskódok könyvtára
|   └─ radar_code.ino       # Az elkészült, kommentelt Arduino C++ forráskód
|
|— /hardware/               # Hardveres tervek és áramkörök
|   └─ circuit_schematic.png # Tinkercad kapcsolási rajz kimentett képe
|   └─ wiring_table.md      # Huzalozási jegyzék táblázatos formában
|
└─ /docs/                   # Hivatalos egyetemi dokumentációk
    └─ projekt_dokumentacio.pdf # Ezen részletes dokumentáció PDF formátumban
```

7.2. Commit stratégia

A fejlesztés során a módosítások nem ömlesztve, hanem logikai mérföldkövenként lettek rögzítve (commitolva), egyértelmű üzenetekkel Bóta Richárd által:

- INITIAL COMMIT: README.md struktúra létrehozása
- FEAT: Szervómotor mozgásának leprogramozása és tesztelése
- FEAT: HC-SR04 távolságmérés integrálása és képlet tesztelése
- FIX: pulseIn timeout hozzáadása a fagyások elkerülésére
- DOCS: Kapcsolási rajz és végleges 12 oldalas dokumentáció feltöltése

8. Fejlesztési lehetőségek és konklúzió

8.1. Jövőbeni fejlesztési és bővítési lehetőségek

Bár a jelenlegi rendszer teljes mértékben kielégíti a Digitális Technika II. tantárgy követelményeit, a moduláris felépítésnek köszönhetően számos ponton bővíthető ipari vagy haladóbb szintre:

- **Processing vizualizáció:** Az Arduino által a soros porton küldött adatok egy PC-n futó Processing program segítségével grafikus, forgó katonai radar-képernyővé alakíthatók, ahol az akadályok piros foltként jelennek meg a zöld koordináta-rendszerben.
- **I2C LCD Kijelző integrálása:** Egy 16x2-es folyadékkristályos kijelző beépítésével a rendszer számítógép nélkül is képes lenne megjeleníteni az aktuális legközelebbi akadály szögét és távolságát, növelve az autonómiát.
- **Hardveres megszakítások (Interrupts):** Az Echo pin figyelése átalakítható lenne hardveres külső megszakítás alapúra (*AttachInterrupt*), így a mikrovezérlőnek nem kellene a *pulseIn()* függvény futása alatt blokkoltan várakoznia, hanem a háttérben egyéb számításokat vagy simább motorfinomításokat is végezhetne.

8.2. Összefoglaló konklúzió

A projekt sikeresen megvalósította a kitűzött célokat. Sikerült egy olyan összetett mikrokontrolleres alkalmazást tervezni, huzalozni és programozni, amely megbízhatóan képes a térpásztázásra és a valós idejű távolságalapú döntéshozatalra. A Tinkercad szimuláció igazolta, hogy a szoftveresen implementált matematikai képletek és szűrőalgoritmusok a valóságban is pontos eredményt adnak. A GitHub verziókezelő használata hozzájárult a strukturált fejlesztési folyamat elsajátításához, amely a későbbi mérnöki munka során is elengedhetetlen kompetenciát jelent.

9. Felhasznált források és irodalomjegyzék

- [1] Drenyovszki R. – Zsupányi K.: *Digitális Technika II. – Logikai kapuáramkör alapok és mikrovezérlők*, Neumann János Egyetem, GAMF Kar, Kecskemét, 2026.
- [2] Arduino LLC: *Arduino Language Reference & Official Documentation*, Elérhetőség: <https://www.arduino.cc/reference/en/>
- [3] Autodesk Inc.: *Tinkercad Circuits Interactive Simulation Platform*, Elérhetőség: <https://www.tinkercad.com/>
- [4] TowerPro Co.: *SG90 Micro Servo Motor Specification and PWM Timing Datasheet*, 2020.
- [5] Electronic Freaks Inc.: *HC-SR04 Ultrasonic Sensor Distance Measurement Waveform and Timing Analysis*, 2018.

Nyilatkozat: Ez a dokumentáció a hallgató önálló munkájának eredménye. A felhasznált külső források, szoftverkönyvtárak és elméleti jegyzetek a forrásjegyzékben pontosan fel lettek tüntetve.

Készítő aláírása: Bóta Richárd s.k. (LUQEV0)